



VotePulse

E-VOTING & ELECTION MANAGEMENT PLATFORM

System Flow, File Structure & Technology Specification

Module Interaction Flow | Complete File System Tree | File Purpose Guide

FRONTEND TIER

React 19 + Vite 8

Single Page App • Glassmorphism CSS

BACKEND TIER

Node.js + Express 4

REST API • Check Middleware Pipeline

DATABASE TIER

MongoDB + Mongoose

NoSQL Store • Resilient JSON Fallback

AUTHENTICATION

6-Digit Gmail OTP

Automated SMTP Verification Delivery

CRYPTOGRAPHY

SHA-256 Ballot Seal

Tamper-Evident Hash Audit Receipts

FOLDER MODEL

Plain-English & Clean

frontend-website • backend-server • db

ACADEMIC PROJECT SUBMISSION DETAILS

Project Title:	VotePulse — Secure E-Voting & Election Management System
Project Type:	Undergraduate Minor Project Submission (CSE/IT)
Architecture Pattern:	3-Tier Decoupled Flow Architecture (UI Website -> Server -> Database)
Platform Scope:	Voter Portal, Admin Console, Candidate Hub & Public Ballot Audit Tool
Deployment Target:	Single Node Production Server (Node.js Express + Vite Bundled Static Assets)
Document Version:	Release 2.3 — Master Architectural Specification & File Tree Diagram

1. Core System Modules & How They Work Together

INTER-MODULE FLOW

VotePulse is structured around 4 distinct functional modules that interact through a central React state controller (App.jsx) on the client and an Express REST API hub on the server. Below is the complete visual map of each module, its operational lifecycle, and how data moves across frontend components, backend APIs, validation checks, helper utilities, and database schemas.

MODULE 1: VOTER PORTAL MODULE

UI File: frontend-website/src/voter/VoterPortal.jsx **API Route:** backend-server/api/voter.routes.js
DB Schema: database/models/voter.schema.js **Services:** checks/validation.js • helpers/email.js (Gmail OTP) • helpers/cipher.js (SHA-256)

How It Works: Voter registers with Name, Voter ID, Email & Password -> validation.js checks syntax -> email.js sends 6-digit OTP via Gmail SMTP -> voter enters OTP to activate account -> logs in with Voter ID -> views active elections -> casts ballot -> cipher.js seals vote with SHA-256 hash -> confetti.js launches celebratory visual animation.

MODULE 2: ADMIN CONSOLE MODULE

UI File: frontend-website/src/admin/AdminConsole.jsx **API Route:** backend-server/api/admin.routes.js & api/election.routes.js
DB Schema: database/models/admin.schema.js & models/election.schema.js **Services:** checks/validation.js (Admin credential verification) • database/index.js

How It Works: Admin logs in with privileged ADM- credentials -> App.jsx unlocks Admin Console -> fetches live election statistics (total registered voters, ballots cast, turnout %) -> renders real-time candidate vote tallies in dynamic bar charts -> creates new elections with start/end schedules -> registers verified candidates -> views voter roster with ADM-9999 master privacy filter.

MODULE 3: CANDIDATE PORTAL MODULE

UI File: frontend-website/src/candidate/CandidatePortal.jsx **API Route:** backend-server/api/candidate.routes.js
DB Schema: database/models/candidate.schema.js **Services:** checks/validation.js (Party emblem validation) • database/models/election.schema.js

How It Works: Prospective candidates or administrators submit nominations -> input candidate name, select political party, and assign party symbol -> backend links candidate to specific active election ID -> database records candidate document -> candidate automatically appears on the voter ballot screen for eligible voters.

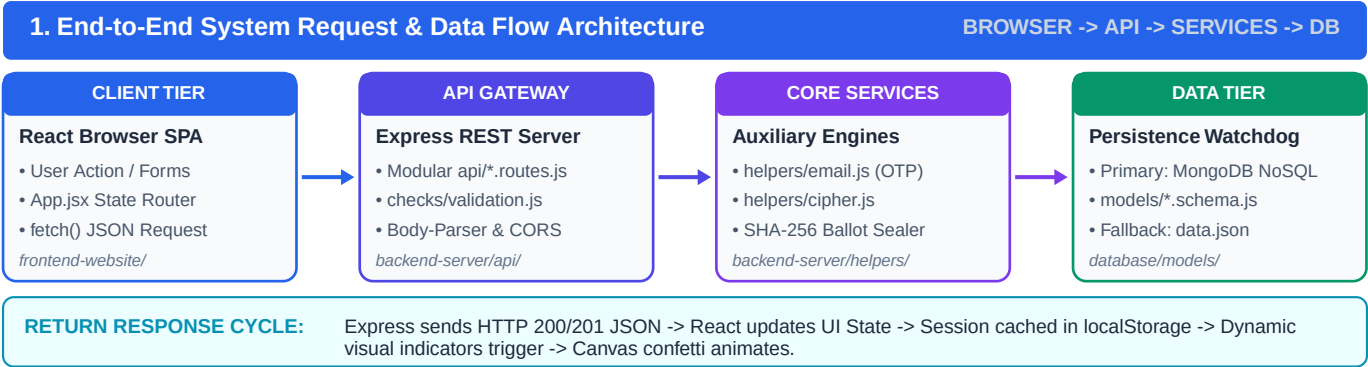
MODULE 4: BALLOT AUDIT & VERIFICATION MODULE

UI File: frontend-website/src/audit/BallotAuditTool.jsx **API Route:** backend-server/api/audit.routes.js
DB Schema: database/models/voter.schema.js (votes collection / data.json) **Services:** helpers/cipher.js (SHA-256 signature verification) • checks/validation.js

How It Works: Independent public audit tool accessible to any citizen -> voter inputs their unique SHA-256 ballot hash receipt -> GET /api/vote/audit/:hash queries the database -> matches the cryptographic hash seal -> displays official verification badge with timestamp, proving the vote was counted without exposing which candidate was selected.

CENTRAL INTEGRATION GLUE (How Modules Communicate):

Client-side: App.jsx maintains the activeTab state and user authentication context, seamlessly mounting the corresponding portal. Server-side: backend-server/api/index.js mounts all module routes onto a single Express server on port 3000.



2. Complete File System Architecture Tree		FULL WORKSPACE REPOSITORY
 E:\Minor project\	Root project directory for VotePulse platform	
 frontend-website/	All client-side user interface source code (React 19 + Vite 8)	
 src/	Modular React source code	
 voter/	Flow 1: Voter registration, OTP verification, authentication & voting (VoterPortal.jsx)	
 admin/	Flow 2: Election operations, live analytics & candidate roster (AdminConsole.jsx)	
 candidate/	Flow 3: Candidate registration and application review (CandidatePortal.jsx)	
 audit/	Flow 4: Cryptographic ballot verification and audit search (BallotAuditTool.jsx)	
 common/	Shared UI components: global header (Navbar.jsx) and modal (SettingsModal.jsx)	
 helpers/	Client utilities: confetti.js (HTML5 Canvas celebration) and formNavigation.js (auto-focus)	
 App.jsx	Central routing controller managing view state, session authentication, and modals	
 main.jsx	React entry point executing ReactDOM.createRoot and mounting root component	
 index.css	Custom glassmorphism design system, CSS variables, dark theme, and fluid layout	
 backend-server/	All server-side business logic, REST APIs, middleware, and email services	
 api/	Modular REST API endpoints aggregated into a unified Express router	
 voter.routes.js	Endpoints: /api/auth/register, /api/auth/verify-otp, /api/auth/login, /api/vote	
 admin.routes.js	Endpoints: /api/admin/stats, /api/admin/voters, /api/admin/voters/:id	
 candidate.routes.js	Endpoints: /api/candidates, /api/candidates/add	
 election.routes.js	Endpoints: /api/elections, /api/elections/create	
 audit.routes.js	Endpoints: /api/vote/audit/:hash (public cryptographic ballot lookup)	
 index.js	Master router mounting all sub-routes into single Express router instance	
 checks/	Request validation middleware (validation.js) inspecting email, OTP, and input payloads	
 helpers/	Server utilities: email.js (Nodemailer Gmail OTP) and cipher.js (SHA-256 + encryption)	
 app.js	Express application instance, middleware registration, static file serving, and port binding	
 database/	Data persistence layer with MongoDB Mongoose models and resilient fallback store	
 models/	Mongoose schemas: voter.schema.js, admin.schema.js, candidate.schema.js, election.schema.js	
 connection.js	MongoDB connection manager with auto-detection of server connectivity	
 index.js	Unified database interface exporting CRUD operations across MongoDB or fallback	
 data.json	Resilient disk storage ensuring zero downtime if MongoDB server is offline	
 docs/	Project documentation, technical reports, presentation decks, and generated PDFs	
 public/	PWA icons: favicon.ico, icon-192.png, icon-512.png, and manifest.json	
 server.js	Root entry point delegating directly to backend-server/app.js	
 vite.config.mjs	Vite build configuration with React plugin and development proxy setup	
 package.json	Project manifest with dependencies, devDependencies, and execution scripts	
 start-server.bat	One-click Windows batch script to launch the production server environment	

1. Frontend File Purpose Breakdown: frontend-website/src/

CLIENT UI MODULES

- src/voter/VoterPortal.jsx** — Voter Registration, OTP & Voting Screen
Complete voter UI: handles registration form, OTP verification modal, login, election candidate cards, and cryptographic ballot receipt. Keeps voter logic isolated.
- src/admin/AdminConsole.jsx** — Election Operations & Analytics Console
Privileged dashboard: live voter turnout %, real-time vote tallies rendered in dynamic bar charts, election scheduling modal, candidate manager, and voter roster with ADM-9999 masking.
- src/candidate/CandidatePortal.jsx** — Candidate Application & Nomination Portal
Allows prospective candidates to submit election nominations, choose party affiliation, select emblems, and view approval status independent of voter forms.
- src/audit/BallotAuditTool.jsx** — Public Cryptographic Ballot Verification
Citizen trust tool: provides an audit search bar where voters enter their SHA-256 ballot hash to verify their vote is in the immutable ledger without exposing choices.
- src/common/Navbar.jsx & SettingsModal.jsx** — Header Navigation & Preferences Modals
Navbar renders top brand header, role indicators, and tab navigation. SettingsModal provides accessible font scaling, high-contrast mode, and dark/light themes.
- src/helpers/confetti.js & formNavigation.js** — Canvas Particle Engine & Accessible Input Navigation
Custom physics engine rendering 60fps canvas celebration particles on ballot submission. formNavigation.js provides automatic focus advancing across multi-digit OTP inputs.
- src/App.jsx, main.jsx, index.css** — State Router, DOM Entry Point & Glassmorphism Design
App.jsx manages activeTab and session auth. main.jsx mounts the React component tree into index.html. index.css establishes custom CSS variables, gradients, and blur cards.

2. Backend & Database File Purpose Breakdown

SERVER & STORAGE ENGINES

- backend-server/api/*.routes.js** — Domain-Separated REST API Controllers
5 route files handling endpoints: voter.routes.js (/register, /login, /vote), admin.routes.js (/stats, /voters), candidate.routes.js, election.routes.js, and audit.routes.js.
- backend-server/checks/validation.js** — Input Sanitization & Request Validation
Security middleware validating email formats, password complexity, OTP expiration, and request bodies before they reach business logic or database queries.
- backend-server/helpers/email.js & cipher.js** — Gmail SMTP Mailer & SHA-256 Hashing
email.js connects via Gmail SMTP (port 587) to automate 6-digit OTP delivery. cipher.js computes tamper-proof SHA-256 ballot hashes and encrypts voter IDs via Caesar cipher.
- backend-server/app.js & server.js** — Express Server Configuration & Port Binding
Configures Express middleware (CORS, JSON parser), mounts /api router, serves production dist/ static files, and binds HTTP listener to port 3000.
- database/models/*.schema.js** — Mongoose NoSQL Schema Definitions
Defines data models: voter.schema.js (voter credentials, OTP & hasVoted flags), admin.schema.js (ADM role access), candidate.schema.js, and election.schema.js.
- database/connection.js, index.js, data.json** — Database Watchdog & Resilient Disk Fallback
connection.js auto-detects MongoDB availability. If offline, index.js seamlessly redirects CRUD queries to local database/data.json, ensuring 100% demo uptime.

1. Programming & Markup Languages		FOUNDATION STACK
JavaScript (ES6+)	Full-Stack Runtime Language	Primary programming language used across both frontend (React) and backend (Node.js). Leverages modern ES6+ features including Promises, async/await, arrow functions, object destructuring, and ES modules on the client alongside CommonJS on the server.
JSX	React UI Syntax Extension	Declarative XML-like syntax extension for JavaScript that defines React component hierarchies. Compiles down to optimized React.createElement() calls via Vite and esbuild, combining HTML layout expressiveness with full JavaScript logic.
CSS3 (Custom Vanilla)	Styling & Design System	Engineered entirely in index.css without third-party CSS utility frameworks like Tailwind. Utilizes CSS Custom Properties (variables), backdrop-filter blur effects for glassmorphism, flexbox/grid layouts, smooth transition curves, and media query breakpoints.
HTML5	Application Shell & Metadata	Defines the single-page application shell (index.html). Provides semantic meta tags, mobile viewport optimization, Google Fonts preconnect links, and Progressive Web App manifest associations.
JSON	Data Interchange & Storage	Standard data exchange format across REST endpoints. Additionally powers local persistent storage (database/data.json), package configuration (package.json), and PWA metadata (manifest.json).

2. Frontend Frameworks & Client Technologies		CLIENT SUBSYSTEM
React 19	Core UI component framework using functional components, state management (useState), side-effects (useEffect), and mutable refs (useRef) in a Single-Page Application (SPA) architecture.	
Vite 8	Next-generation frontend tooling offering instant dev-server startup, lightning-fast Hot Module Replacement (HMR), and highly optimized production Rollup/esbuild bundling.	
Lucide React	Modern, ultra-clean SVG icon package providing crisp, scalable iconography (Vote, Shield, CheckCircle, Settings, Logout) with zero performance impact.	
Accessibility Engine	Comprehensive keyboard accessibility, focus trapping in modal dialogs, and dynamic font scale preferences managed via SettingsModal.	
HTML5 Canvas Confetti	Custom physics animation in helpers/confetti.js rendering 60 FPS celebratory particle physics during ballot confirmation without external library dependencies.	
PWA (Service Worker)	sw.js provides intelligent offline asset caching and rapid reload speeds, paired with manifest.json for full "Add to Home Screen" mobile installation support.	

3. Backend Server Technologies & Runtime		SERVER SUBSYSTEM
Node.js v24	Asynchronous, non-blocking V8 event-driven runtime handling concurrent HTTP requests and managing database input/output pipelines.	
Express.js 4	Industry-standard web framework providing structured routing, JSON body-parsing middleware, CORS management, and static bundle delivery.	
Nodemailer 9	SMTP mail transmission library configured for Gmail STARTTLS (port 587) to automate the dispatch of 6-digit OTP verification security tokens.	
dotenv 17	Environment variable manager isolating secrets (MONGO_URI, EMAIL_USER, EMAIL_PASS) into .env, preventing credential leakage in git history.	
Built-in Crypto	Node.js native cryptographic engine generating secure 256-bit SHA-256 ballot hashes for tamper-proof vote verification.	
Checks Middleware	Dedicated validation layer (checks/validation.js) sanitizing email syntax, credential lengths, and request payloads before route execution.	

1. Database Architecture & Mongoose Schemas

PERSISTENCE ENGINE

voter.schema.js

voters Collection

Fields: voterID (encrypted), name, email, password, isVerified, otp, otpExpires, hasVoted, votedAt

Purpose: Tracks voter lifecycle from unverified registration through OTP activation and ballot cast status.

admin.schema.js

admins Collection

Fields: adminID (must start with ADM-), password, role ("superadmin"), createdAt

Purpose: Controls administrative access. ADM-9999 master maintenance accounts are filtered from general lists.

candidate.schema.js

candidates Collection

Fields: name (String), party (String), symbol (Emblem string), electionId (Ref), voteCount (Number)

Purpose: Stores registered candidates linked to elections. Vote counts update in real-time as ballots are sealed.

election.schema.js

elections Collection

Fields: title, description, startDate, endDate, status ("active" | "upcoming" | "closed")

Purpose: Manages election schedules and voting windows. Only active elections accept vote submissions.

2. High-Availability Resilient Failover Engine

ZERO-CRASH TOLERANCE

AUTOMATIC DATABASE WATCHDOG & LOCAL FALLBACK STORE

VotePulse features an intelligent auto-detecting database watchdog in database/connection.js. At application startup, the system attempts a connection to the primary MongoDB service (mongodb://127.0.0.1:27017/votepulse). If MongoDB is offline or connection fails (ECONNREFUSED), the database layer automatically falls back to an in-memory repository backed by atomic disk persistence in database/data.json. This ensures the application remains 100% operational during offline presentations, defense evaluations, or server disruptions.

3. Comprehensive Security & Cryptography Matrix

SECURITY ARCHITECTURE

6-Digit Email OTP

Nodemailer dispatches single-use OTPs with 10-minute expiration to prevent fake or sybil registrations.

SHA-256 Ballot Sealing

Each ballot is sealed with a unique 256-bit cryptographic hash returned to the voter as an audit receipt.

Reversible Voter Cipher

Voter IDs are obfuscated via reversible substitution cipher in cipher.js before being stored on disk.

One-Vote Enforcement

Strict database validation checks hasVoted flags; second vote attempts trigger 403 Forbidden.

Master Admin Privacy

Admin accounts with IDs beginning ADM-9999 are excluded from voter lists to prevent credential harvesting.

Strict Input Validation

Request payloads pass through checks/validation.js to protect against SQL/NoSQL injection and payload overflow.

1. Development Environment & Tooling Ecosystem

DEVELOPMENT TOOLCHAIN

VS Code
Primary Code Editor
Used for source editing, JSX highlighting, and Git workflow.

MongoDB Compass
Database GUI
Visual inspection tool to query collections, schemas, and live votes.

npm 10+
Package Manager
Automates dependency installs, build scripts, and execution triggers.

Chrome DevTools
Testing & Profiling
Inspects DOM hierarchy, network payloads, application storage, and console logs.

Git & GitHub
Version Control
Tracks code modifications with .env protection via .gitignore.

start-server.bat
Windows Launcher
One-click Windows batch script for automated startup without CLI.

2. Complete npm Packages & Dependencies (package.json)

DEPENDENCY AUDIT

PACKAGE NAME	VERSION	TYPE	ROLE & ARCHITECTURAL PURPOSE
react	^19.2.8	dependency	Core declarative UI library powering the modular component tree
react-dom	^19.2.8	dependency	Browser DOM rendering engine and event system adapter for React
express	^4.19.2	dependency	High-performance web application framework powering REST API server
mongoose	^9.9.3	dependency	Asynchronous MongoDB Object Document Modeling library with schema validation
nodemailer	^9.0.5	dependency	Enterprise email transmission library delivering Gmail SMTP OTPs
cors	^2.8.5	dependency	Enables secure Cross-Origin Resource Sharing across Vite and Express
dotenv	^17.4.2	dependency	Injects environment secrets from .env file into process.env at startup
lucide-react	^1.32.0	dependency	Lightweight, modern SVG icon library for seamless interface visuals
vite	^8.2.1	devDependency	Next-generation ultra-fast frontend build tool and development server
@vitejs/plugin-react	^6.0.5	devDependency	Official Vite plugin enabling React fast refresh and JSX compilation
esbuild	^0.28.2	devDependency	High-speed JavaScript and TypeScript bundler backing Vite builds
pdfkit	^0.20.2	devDependency	Declarative programmatic PDF generation engine creating this report

3. Execution Commands & Operational Scripts

NPM COMMAND REFERENCE

npm start	Starts the production server on port 3000 (delegates to server.js -> backend-server/app.js)
npm run dev	Launches Vite development server with Hot Module Replacement on port 5173
npm run build	Executes Vite production build, outputting optimized bundles to dist/ directory
start-server.bat	Windows batch executable for one-click server launch without manual command entry